

Student Name: Ya Zou

Email: zouy2@oregonstate.edu

Project Number: #5

Project Name: CUDA: Monte Carlo Simulation

Project Report #5

1. Tell what machine you ran this on

Rabbit server

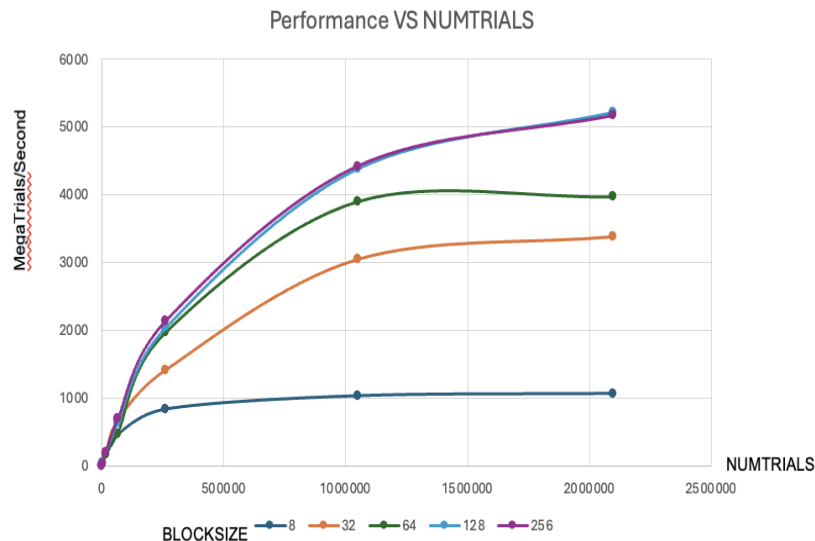
2. What do you think this new probability is?

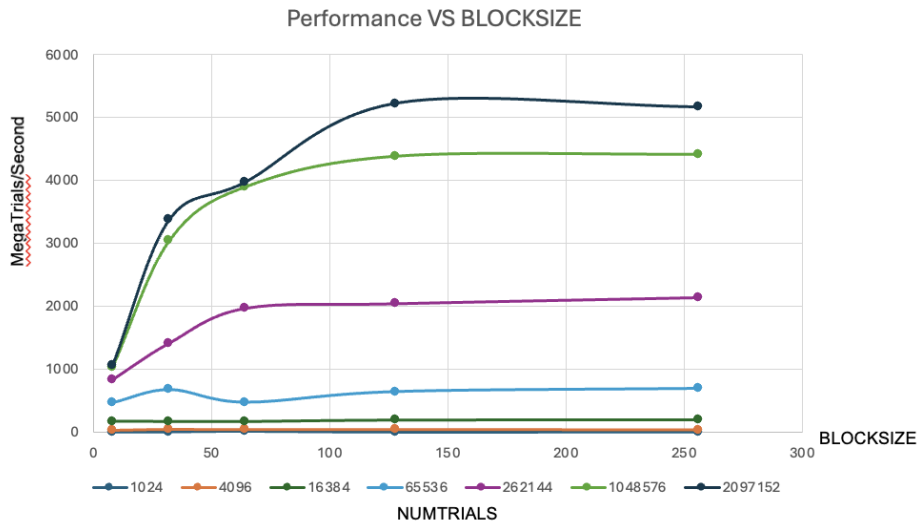
The probability that I got is 83.82.

3. Show the rectangular table and the two graphs

Performance table:

Sum of MegaTrials/Second	BLOCKSIZE				
NUMTRIALS	8	32	64	128	256
1024	8.3182	9.6241	12.4708	8.1695	8.8938
4096	32.6281	48.7991	45.4062	48.1022	40.0877
16384	175.2225	172.3325	170.326	198.2191	202.692
65536	470.5882	679.2703	478.3929	647.2819	698.2612
262144	842.9718	1420.4959	1969.2308	2043.9123	2140.5801
1048576	1037.356	3045.3532	3898.6318	4388.3756	4420.3427
2097152	1071.6377	3378.4926	3971.1566	5224.49	5165.1956





4. What patterns are you seeing in the performance curves?

The patterns observed from the performance graphs indicate that for each block size value, performance increases as the number of trials (NUMTRIALS) increases, with this growth being more significant for larger block sizes. The curves suggest that performance continues to improve with more trials but at a decreasing rate. Larger block sizes, such as 128 and 256, show higher performance compared to smaller block sizes like 8 and 32, with the performance gap becoming more evident as the number of trials increases. For larger block sizes, performance continues to increase even at high NUMTRIALS, though the rate of increase slows down, while for smaller block sizes, performance gains taper off earlier, indicating a more rapid approach to saturation. Additionally, for each NUMTRIALS value, there's a noticeable increase in performance as the block size increases from the smallest value to a certain point, after which the performance curves flatten out, indicating diminishing returns. At high NUMTRIALS values, such as 1048576 and 2097152, performance peaks and then slightly declines or stabilizes. Larger NUMTRIALS values tend to show higher peak performance, suggesting that the system handles more trials per second with increased trials up to a point, whereas smaller NUMTRIALS values reach lower peak performance and flatten out quickly.

5. Why do you think the patterns look this way?

When the block size is insufficiently large(4 or 8), it can result in the GPU's processing power being underutilized. This occurs as there are more inactive processing elements within the compute units, leading to a considerable amount of compute time being wasted. Moreover, the overhead involved in configuring the GPU for computation may not be sufficiently offset by the workload processed on the GPU. This discrepancy between setup overhead and actual computation can hamper the overall performance and effectiveness of the system. Hence, selecting an appropriate block size is paramount to maximizing the utilization of the GPU's resources.

6. Why is a BLOCKSIZE of 8 so much worse than the others?

The first reason is that each thread in a block has a unique ID, which is used to select work and pull the right data from memory. If the block size is not a multiple of the warp size (32 threads), some threads might be idle during each warp execution, leading to inefficient utilization of the GPU's computational resources. In CUDA, a warp (a group of 32 threads) executes the same instruction simultaneously on different data. If a warp's execution needs to stop due to memory access, having multiple warps ready to run can prevent the execution from going idle. A block size that is not a multiple of the warp size might lead to warps that are not fully populated, reducing the effectiveness of this warp-switching mechanism. Also, each thread has access to different types of memory with varying access speeds and scopes. The block size can influence how effectively these memory types are utilized. For example, smaller blocks might not fully utilize the available shared memory, leading to more global memory accesses which are slower. The number of threads per block should ideally be a multiple of the warp size (32). This is because all threads in a warp execute the same instruction at any given time but on different data. Threads in different warps will usually be executing different instructions at any given time. Therefore, having 192 or 256 (multiples of 32) threads per block can lead to more efficient execution.

7. How do these performance results compare with what you got in Project #1? Why?

These performance results are much better than those from Project 1. In Project 1, we saw performance improvements with OpenMP as we increased the number of threads from 1 to 8. However, the gains leveled off after a certain point because of the overhead of managing threads and the limited number of CPU cores. CPU parallelization is less efficient for large-scale simulations that need a

lot of iterations and data processing. This is because OpenMP uses CPU cores for parallelism, which is effective but limited by the number of cores in the CPU. In contrast, CUDA uses the GPU's massively parallel architecture, which has thousands of smaller, more efficient cores that can handle many threads at once. As a result, the CUDA implementation provided faster computation times and better scalability as the number of threads increased. This shows the advantage of using GPU parallel computing for intensive tasks like Monte Carlo simulations.

8. What does this mean for what you can do with GPU parallel computing?

Based on the experiments, a local work size of 128 appears to be optimal. However, processing small data sizes on the GPU may not be worthwhile, as the overhead of setting up may outweigh the benefits. GPU parallel computing becomes advantageous only with larger data sizes, where it can efficiently handle the setup overhead and deliver notable performance gains. Therefore, we have to choose the proper block size for GPU parallel programming to maximize computational performance. We can also follow the thread rule that 192 or 256 are good numbers of threads per block (multiples of the warp size).